

Saloni Aryal

Prg:400

Week 1 Assignment

Object-oriented programming (OOP)

Object-Oriented Programming (OOP) is a programming paradigm which codes are structured into classes and objects. It is concerned with the modelling of real world objects in a program like students, cars, or bank accounts. A class is a template that describes the characteristics (attributes) and actions (methods) of an object. An object is an example of a class which includes real data and has functions in accordance with the class. OOP also assists programmers in creating easy to write code that can be easily reused and maintained.

Encapsulation is one of the significant concepts of OOP that implies concealing internally stored data and regulating access to it using methods. This assists in guarding information against unintentional alteration. The other concept is that of inheritance where a new class (child class) is allowed to inherit properties and methods of an existing class (parent class). This facilitates reuse of code and minimizes repetition. Another important concept is polymorphism which enables the same method name to be executed differently by the object calling it.

Encapsulation

One of the major principles of Object-Oriented Programming is encapsulation. It means that the data (variables) and the methods (functions) are bundled within a class and that only a few of the internal details of the object are directly accessible. Simply put, encapsulation assists in securing the internal condition of a particular object and it also assures that information is read or altered only by means of a controlled access.

In Python, the encapsulation is most commonly achieved through the use of private variables in which addition of two underscores (__) at the beginning is added followed by the name of the variable. These are private variables, which one cannot retrieve outside the class. Rather, the class offers the public means through which the value can be accessed or changed, i.e., through the methods like getters and setters. This will enhance the security of the data, avoid unintended alteration and serious management of data usage.

Considering the example of a bank account system, other portions of the program should not be allowed to directly alter the account balance. When an individual is able to alter the balance directly, he or she may result in incorrect or unsafe operations. Rather, it is only by

using certain procedures such as deposit, withdraw or get balance that the balance is accessed. These methods make sure that one does not commit a mistake in rules like checking on adequate balance before doing any operations.

Through encapsulation, the programmers are able to ensure that they are able to secure sensitive data, code reliability and make programs more manageable and easier to maintain.

```
In [1]: class BankAccount:

    # Constructor
    def __init__(self, account_holder, balance):
        self.account_holder = account_holder
        self.__balance = balance    # Private variable

    # Method to deposit money
    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount
            print("Amount deposited successfully.")
        else:
            print("Invalid deposit amount.")

    # Method to withdraw money
    def withdraw(self, amount):
        if amount <= self.__balance:
            self.__balance -= amount
            print("Withdrawal successful.")
        else:
            print("Insufficient balance.")

    # Getter method to access private balance
    def get_balance(self):
        return self.__balance

# Creating object
account1 = BankAccount("Saloni", 1000)

# Using methods to access and modify balance
account1.deposit(500)
account1.withdraw(300)

# Accessing balance through getter
print("Current Balance:", account1.get_balance())
```

Amount deposited successfully.

Withdrawal successful.

Current Balance: 1200

The result is constructed step by step as the methods of the BankAccount class are called. To begin with, when the object `account1 = BankAccount(Saloni, 1000)` is created, the balance holder Saloni is assigned by the constructor of the object, namely, the method **init** vs. set the balance of the bank account, i.e., the account balance, to 1000. Then, the procedure

account1.deposit(500) will be invoked, which will verify whether the sum of the deposit is more than 0. The fact that it is 500 means that it will add 500 to the balance making the new balance 1500 and will print out the Amount deposited successfully. Then, account1.withdraw(300) is run. The program under consideration verifies whether 300 is smaller or equal to the current balance (1500). As it is permitted, 300 is subtracted to the balance, and 1200 is left and the message with the output: Withdrawal successful. is printed. Lastly, the approach get balance returns the current private balance which is 1200 and the program gives the current Balance 1200.

Inheritance

One of the primary notions of Object-Oriented Programming is inheritance. It enables a new class (child class) to re-use the properties and methods of an existing class (parent class). This is because automatically the child class inherits the attributes of the parent class without the need of writing again the same code. The inheritance prevents the duplication of the codes and simplifies the maintenance and the further expansion of the programs.

Python has a kind of inheritance that involves the use of the parent name of the class in the parenthesis when the child class is declared. All the methods and attributes of the parent class are usable by the child class which can add new methods or can alter the already existing methods.

To take an example, when we develop a generalized category such as Animal, it may possess general behaviors such as producing a sound. We can then develop another class extending Animal which is Dog. A dog is an animal just like the Animal and therefore the Dog class can utilize the methods of the Animal one and also possesses its own behavioral characteristics like barking.

Inheritance establishes a hierarchical connectivity between classes with general features being defined in the parent class with further specific features being added in the child class.

```
In [3]: # Parent class
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        print("Animal makes a sound")

# Child class inheriting from Animal
class Dog(Animal):
    def bark(self):
        print(self.name, "barks")
```

```
# Another child class inheriting from Animal
class Cat(Animal):
    def meow(self):
        print(self.name, "meows")

# Creating objects
dog1 = Dog("Buddy")
cat1 = Cat("Kitty")

# Using inherited method
dog1.speak()
cat1.speak()

# Using child class methods
dog1.bark()
cat1.meow()
```

Animal makes a sound
Animal makes a sound
Buddy barks
Kitty meows

In running the program, it initially forms the parent class Animal which holds the name of the animal and has a method speak which displayed Animal makes a sound. Then there are two child classes of the Animal class: Dog and Cat, which inherit all the characteristics of the members, i.e. the name attribute and the speak() method. Then two objects are made: dog1 = Dog(Buddy) and cat1 = Cat(Kitty), therefore the names Buddy and Kitty are held in the memory. In the case of calling dog1.speak and cat1.speak, the two objects call the inherited method of the Animal class and the Animal makes a sound. The program then calls dog1.bark and cat1.meow methods of the child class. The bark() method print out; Buddy barks and the meow() method print out; Kitty meows. Thus the final product will display the overall sound of the animal and then the precise sounds of the dog and the cat.

Polymorphism

Polymorphism refers to a significant concept in the Object-Oriented Programming which implies many forms. It enables the same name of a method to be used in performing various actions based on the object calling it. That is, classes may contain methods of the same name, yet the behaviour of those methods may vary.

Polymorphism will enhance the flexibility as well as the extendability of programs since only one interface will be able to operate with different objects. The use of method overriding, where a child class has a different implementation of a method that is already present in the parent class, is commonly used in the Python language as a form of polymorphism.

To take one example, we may have a general class, Person, in a hospital system. Based on this course, we are able to develop Doctor and Patient classes. All the classes can have a

method named `display_info` though the information displayed is different. An object representing the doctor may reflect the doctor specialization whereas an object representing a patient may reflect the disease. Although the name of the method is similar, the result varies with the object.

This is the polymorphism concept where the same method name can act differently on the different objects.

```
In [4]: # Parent class
class Person:
    def __init__(self, name):
        self.name = name

    def display_info(self):
        print("This is a person.")

# Child class Doctor
class Doctor(Person):
    def __init__(self, name, specialization):
        super().__init__(name)
        self.specialization = specialization

    # Overriding method (Polymorphism)
    def display_info(self):
        print("Doctor Name:", self.name)
        print("Specialization:", self.specialization)

# Child class Patient
class Patient(Person):
    def __init__(self, name, disease):
        super().__init__(name)
        self.disease = disease

    # Overriding method (Polymorphism)
    def display_info(self):
        print("Patient Name:", self.name)
        print("Disease:", self.disease)

# Creating objects
doctor1 = Doctor("Dr. Sharma", "Cardiologist")
patient1 = Patient("Riya", "Fever")

# Calling the same method for different objects
doctor1.display_info()
print()
patient1.display_info()
```

Doctor Name: Dr. Sharma
Specialization: Cardiologist

Patient Name: Riya
Disease: Fever

When the program is executed, it initially creates a parent class Person which contains the name of a person and contains a method display info which simply prints a generic statement. Then the Person class has two child classes namely Doctor and Patient. Both classes of children inherit one another by use of `super().__init__(name)` whereby in the parent class, the name attribute is passed to the child, as well as the children have their own attributes; the doctor and the disease respectively. The children classes override display info method, meaning that the children classes will offer their alternative to the same method. Then, two objects are formed doctor1 whose name is Dr. Sharma and specialization is Cardiologist, and patient1 whose name is Riya and disease Fever. On calling doctor1.display_info(), the name and specialization of the doctor are printed. Upon calling patient1.display_information(), it displays the name and the disease of the patient. Although the call to the same name display_info() is applied to both the same objects, they will give different results, which bears the concept of polymorphism.

Conclusion

This program illustrates the fundamental principles of Object-Oriented Programming (OOP) which are encapsulation, inheritance, and polymorphism. Encapsulation provides security to the internal data of a class and also restricts access using methods. Inheritance enables child classes to share attributes and methods with a parent class and minimize the duplication of the code and establish a clear hierarchical structure. Polymorphism allows the same method name to act in different ways with the object thus the program becomes more flexible and simpler to extend. A combination of these OOP principles assists in organizing the code, enhancing the degree of reuse as well as making programs more efficient and maintainable.